

## 3. Script custom

En este apartado vamos a crear el script personalizado que leerá las alertas de Wazuh y las enviará por correo a través de Postfix, además de asignarle los permisos necesarios para que Wazuh pueda ejecutarlo correctamente.

## Índice

- 3.1. Crear script
- 3.2. Permisos

### 3.1.Crear script

En el servidor que aloja Wazuh editamos este archivo:

```
sudo nano /var/ossec/integrations/custom-email-postfix
```

(La documentación oficial de Wazuh informa de que el script custom debe crearse aquí)

Dentro de él, agregamos este contenido:

```
#!/usr/bin/env python3

"""
Este script permite enviar las alertas de Wazuh a nuestro correo.

Es una alternativa muy útil en situaciones donde el método tradicional
de envío de alertas por correo no funciona bien, como por ejemplo
cuando wazuh-maild no está entregando las alertas a Postfix y está dando
problemas.

El script recibe una alerta JSON de Wazuh, extrae los datos más importantes
y envía un correo usando sendmail/Postfix.
"""

# Librerías que usamos:
# - json: para leer la alerta en formato JSON
# - subprocess: para ejecutar sendmail
# - sys: para leer el argumento que pasa Wazuh
from email.message import EmailMessage
import json
```

```
import subprocess
import sys

# Correo de origen y destino
FROM_EMAIL = "ESCRIBE_TU_CORREO_ORIGEN"
TO_EMAIL = "ESCRIBE_TU_CORREO_DESTINATARIO"

# Ruta del binario sendmail que usará Postfix
SENDMAIL = "/usr/sbin/sendmail"

# Esta función sirve para leer campos del JSON sin que el script falle
# si falta alguna clave.
def safe_get(data, path, default="-"):
    current = data
    for key in path:
        if not isinstance(current, dict) or key not in current:
            return default
        current = current[key]
    return current

# Esta función convierte listas en texto separado por comas.
def list_to_text(value):
    if isinstance(value, list):
        return ", ".join(str(x) for x in value)
    return str(value)

# Esta función limpia saltos de línea escapados para que el correo se vea
bien.
def clean_text(value):
    if value in (None, "-"):
        return "-"
    if not isinstance(value, str):
        value = str(value)
    return value.replace("\\r\\n", "\n").replace("\r\n", "\n").replace("\r",
"\n").strip()

def main():
    # Wazuh pasa como argumento la ruta del archivo de alerta
    if len(sys.argv) < 2:
        sys.exit(1)

    # Guardamos la ruta del JSON de alerta
```

```

alert_path = sys.argv[1]

# Abrimos y leemos la alerta en formato JSON
with open(alert_path, "r", encoding="utf-8") as f:
    alert = json.load(f)

# Sacamos los datos principales de la alerta
rule_id = safe_get(alert, ["rule", "id"])
rule_level = safe_get(alert, ["rule", "level"])
description = clean_text(safe_get(alert, ["rule", "description"]))
agent_name = safe_get(alert, ["agent", "name"], "manager")
agent_id = safe_get(alert, ["agent", "id"], "000")
agent_ip = safe_get(alert, ["agent", "ip"])
manager_name = safe_get(alert, ["manager", "name"])
full_log = clean_text(safe_get(alert, ["full_log"]))
timestamp = safe_get(alert, ["timestamp"])
groups = list_to_text(safe_get(alert, ["rule", "groups"], []))
location = safe_get(alert, ["location"])
decoder = safe_get(alert, ["decoder", "name"])
alert_id = safe_get(alert, ["id"])

# Datos técnicos del evento alertado
provider = safe_get(alert, ["data", "win", "system", "providerName"])
event_id = safe_get(alert, ["data", "win", "system", "eventID"])
channel = safe_get(alert, ["data", "win", "system", "channel"])
computer = safe_get(alert, ["data", "win", "system", "computer"])
process_id = safe_get(alert, ["data", "win", "system", "processID"])
thread_id = safe_get(alert, ["data", "win", "system", "threadID"])
event_record_id = safe_get(alert, ["data", "win", "system",
"eventRecordID"])
severity = safe_get(alert, ["data", "win", "system", "severityValue"])
system_time = safe_get(alert, ["data", "win", "system", "systemTime"])

# Mensaje del evento alertado
event_message = clean_text(safe_get(alert, ["data", "win", "system",
"message"]))

# Construimos el asunto del correo
subject = f"Wazuh alerta | Nivel {rule_level} | Regla {rule_id} |
{agent_name}"

# Construimos el cuerpo del correo con la plantilla nueva
body = f""ALERTA DE WAZUH

```

## ÍNDICE:

1. Resumen General
2. Regla detectada
3. Datos técnicos del evento alertado
4. Mensaje del evento alertado
5. Log entero

### 1. RESUMEN GENERAL

Fecha: {timestamp}

Agente: {agent\_name} (ID: {agent\_id})

IP del agente: {agent\_ip}

Manager: {manager\_name}

Grupos: {groups}

Ubicación: {location}

Decoder: {decoder}

ID de alerta: {alert\_id}

### 2. REGLA DETECTADA

Regla: {rule\_id}

Nivel: {rule\_level}

Descripción:

{description}

### 3. DATOS TÉCNICOS DEL EVENTO ALERTADO

Provider: {provider}

EventID: {event\_id}

Channel: {channel}

Computer: {computer}

ProcessID: {process\_id}

ThreadID: {thread\_id}

```
EventRecordID: {event_record_id}
Severity: {severity}
SystemTime: {system_time}
```

#### 4. MENSAJE DEL EVENTO ALERTADO

```
{event_message}
```

#### 5. LOG ENTERO

```
{full_log}
"""

# Creamos el mensaje de correo
msg = EmailMessage()
msg["From"] = FROM_EMAIL
msg["To"] = TO_EMAIL
msg["Subject"] = subject
msg.set_content(body)

# Enviamos el correo a través de sendmail/Postfix
subprocess.run(
    [SENDMAIL, "-t", "-oi"],
    input=msg.as_bytes(),
    check=True
)

# Punto de entrada principal del script
if __name__ == "__main__":
    main()
```

⚠️ Ojo: En el script debemos de rellenar estos campos:

```
FROM_EMAIL = "ESCRIBE_TU_CORREO_ORIGEN"
TO_EMAIL = "ESCRIBE_TU_CORREO_DESTINATARIO"
```

## 3.2.Permisos

Le asignamos los permisos necesarios ejecutando estos dos comandos:

```
sudo chmod 750 /var/ossec/integrations/custom-email-postfix  
sudo chown root:wazuh /var/ossec/integrations/custom-email-postfix
```

El primer comando da permisos de lectura y ejecución al dueño y al grupo, y quita acceso al resto.

El segundo comando pone como dueño a `root` y como grupo a `wazuh`, que es como Wazuh espera encontrar ese script.